

```

%% =====
% Design and Analysis of a Butterworth IIR Low-Pass Filter
% =====
% This script:
% 1. Designs a Butterworth IIR LPF from given specifications
% 2. Finds the minimum filter order needed
% 3. Plots magnitude & phase response
% 4. Plots pole-zero diagram
% 5. Plots impulse and step response
% 6. Applies the filter to a noisy test signal
% =====

```

```
clear; clc; close all;
```

%% 1. Filter Specifications

```

Fs = 1000;      % Sampling frequency (Hz)
Fpass = 100;    % Passband edge frequency (Hz)
Fstop = 200;    % Stopband edge frequency (Hz)
Rp = 1;         % Max passband ripple (dB)
Rs = 40;        % Min stopband attenuation (dB)

```

```
% Normalize frequencies to Nyquist (Fs/2)
```

```

Wp = Fpass / (Fs/2);
Ws = Fstop / (Fs/2);

```

%% 2. Determine Minimum Order and Cutoff Frequency

```

[n, Wn] = buttord(Wp, Ws, Rp, Rs);
fprintf('Minimum Butterworth filter order required : %d\n', n);

```

```
Minimum Butterworth filter order required : 7
```

```
fprintf('Normalized cutoff frequency (Wn) : %.4f\n', Wn);
```

```
Normalized cutoff frequency (Wn) : 0.2291
```

%% 3. Design the Butterworth IIR Filter (transfer function form)

```
[b, a] = butter(n, Wn, 'low');
```

```
fprintf('\nNumerator coefficients (b):\n'); disp(b);
```

```

Numerator coefficients (b):
    0.0002    0.0014    0.0042    0.0071    0.0071    0.0042    0.0014    0.0002

```

```
fprintf('Denominator coefficients (a):\n'); disp(a);
```

```

Denominator coefficients (a):
    1.0000   -3.7738    6.5614   -6.6518    4.2030   -1.6437    0.3666   -0.0359

```

%% 4. Magnitude and Phase Response

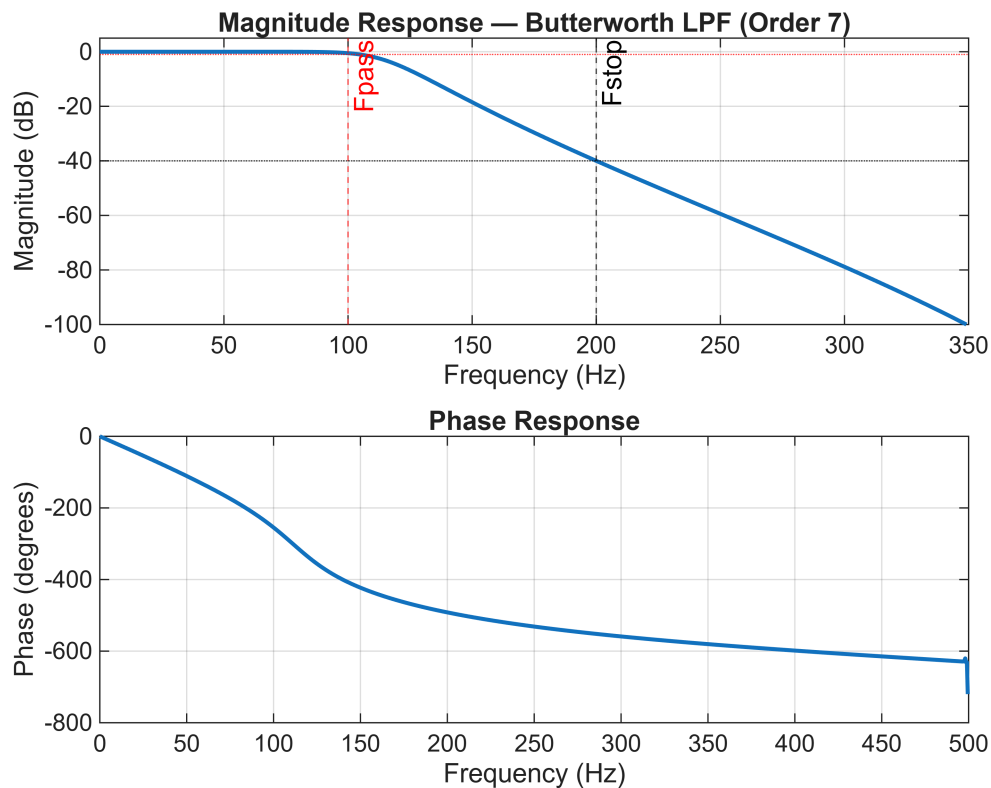
```
[H, f] = freqz(b, a, 1024, Fs);
```

```

figure('Name','Frequency Response');
subplot(2,1,1);
plot(f, 20*log10(abs(H)), 'LineWidth', 1.5);
grid on;
xline(Fpass, '--r', 'Fpass');
xline(Fstop, '--k', 'Fstop');
yline(-Rp, ':r');
yline(-Rs, ':k');
title(sprintf('Magnitude Response – Butterworth LPF (Order %d)', n));
xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');
ylim([-100 5]);

subplot(2,1,2);
plot(f, unwrap(angle(H))*180/pi, 'LineWidth', 1.5);
grid on;
title('Phase Response');
xlabel('Frequency (Hz)'); ylabel('Phase (degrees)');

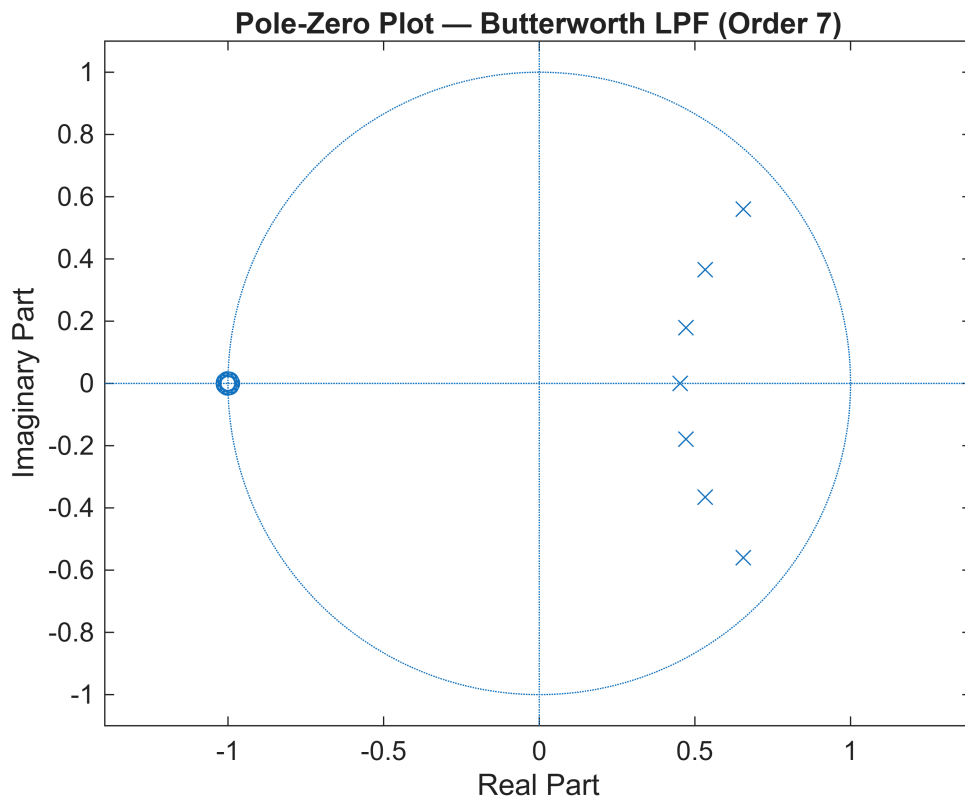
```



```

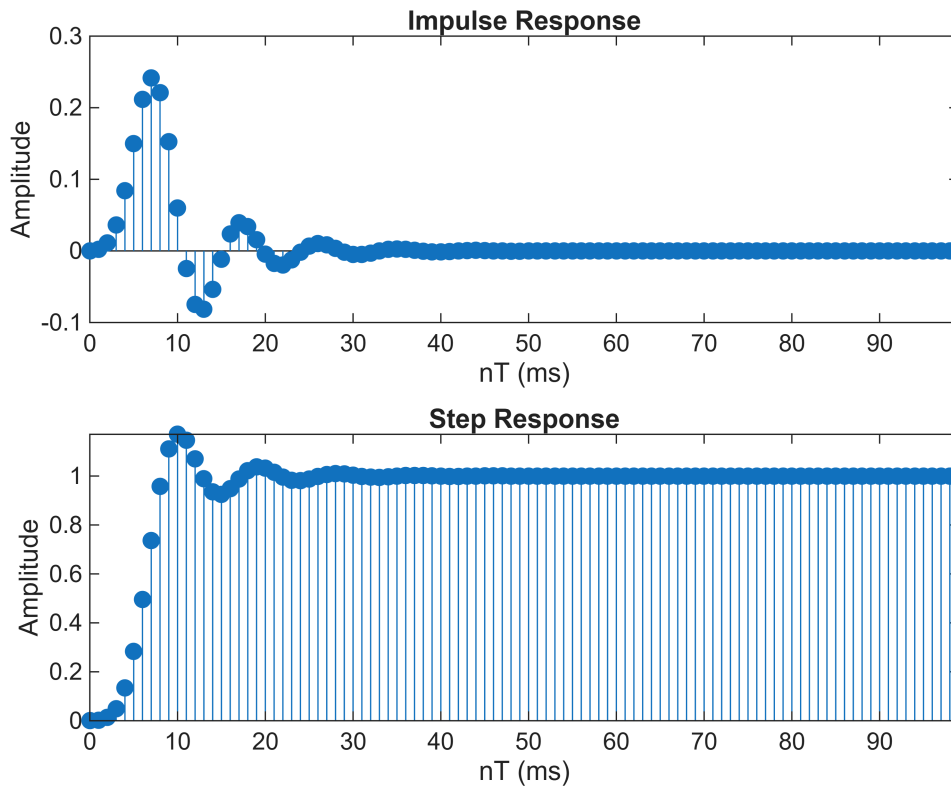
%% 5. Pole-Zero Plot (check stability: all poles inside unit circle)
figure('Name','Pole-Zero Plot');
zplane(b, a);
title(sprintf('Pole-Zero Plot – Butterworth LPF (Order %d)', n));

```



```
%% 6. Impulse and Step Response
figure('Name','Time Domain Response');
subplot(2,1,1);
impz(b, a, 100, Fs);
title('Impulse Response');

subplot(2,1,2);
stepz(b, a, 100, Fs);
title('Step Response');
```



```

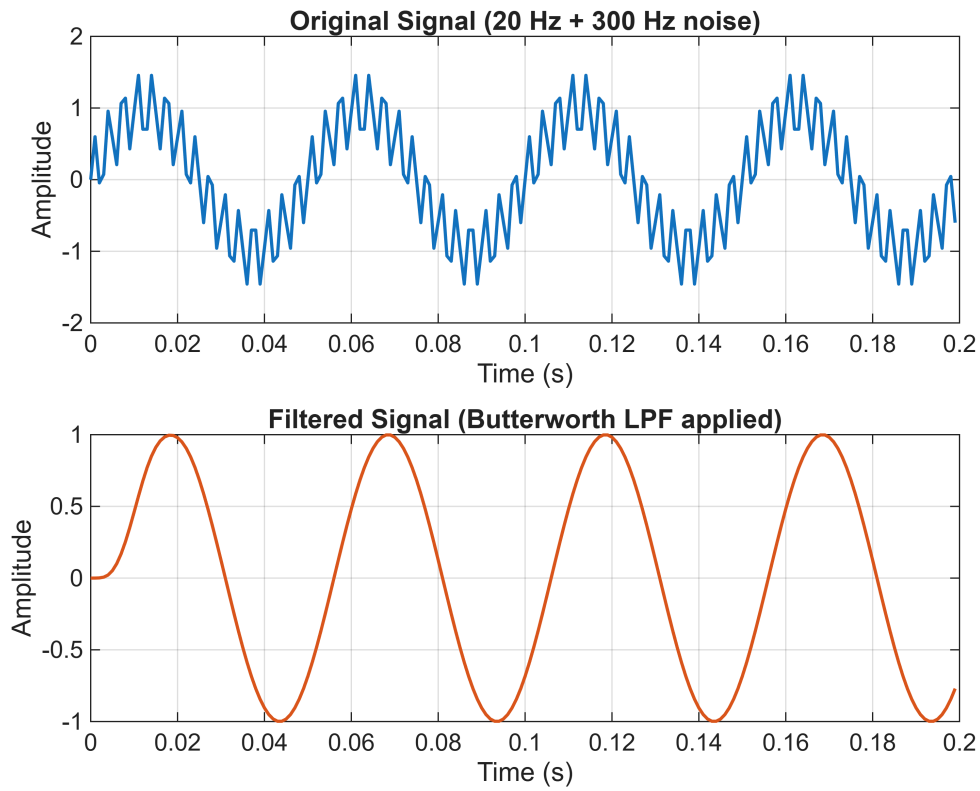
%% 7. Apply Filter to a Test Signal (e.g., signal + high-frequency noise)
t = 0:1/Fs:1-1/Fs;
f1 = 20;      % desired low-frequency component (Hz)
f2 = 300;     % unwanted high-frequency noise (Hz)

x = sin(2*pi*f1*t) + 0.5*sin(2*pi*f2*t); % composite signal
y = filter(b, a, x);                     % apply Butterworth LPF

figure('Name','Filtering Demonstration');
subplot(2,1,1);
plot(t(1:200), x(1:200), 'LineWidth', 1.2);
grid on;
title('Original Signal (20 Hz + 300 Hz noise)');
xlabel('Time (s)'); ylabel('Amplitude');

subplot(2,1,2);
plot(t(1:200), y(1:200), 'LineWidth', 1.2, 'Color', [0.85 0.33 0.1]);
grid on;
title('Filtered Signal (Butterworth LPF applied)');
xlabel('Time (s)'); ylabel('Amplitude');

```



```
% 8. Stability Check
poles = roots(a);
if all(abs(poles) < 1)
    disp('Filter is STABLE (all poles lie inside the unit circle).');
else
    disp('Filter is UNSTABLE (poles found outside the unit circle).');
end
```

Filter is STABLE (all poles lie inside the unit circle).